Artificial Intelligence and Machine Learning

Logistic Regression/Classification

Outline

- Linear Regression (Review)
- Intro to Classification
- Logistic Regression (Linear Classifier)
- Classification Metrics

Recap

Linear Regression

$\hat{y} = mx + b$ (θ_0)

θ (w_0)

w

$J(\theta)$ = closed form

$J(m, b)$ = (Ground truth)

Polynomial

$x_2 = x_1^2$

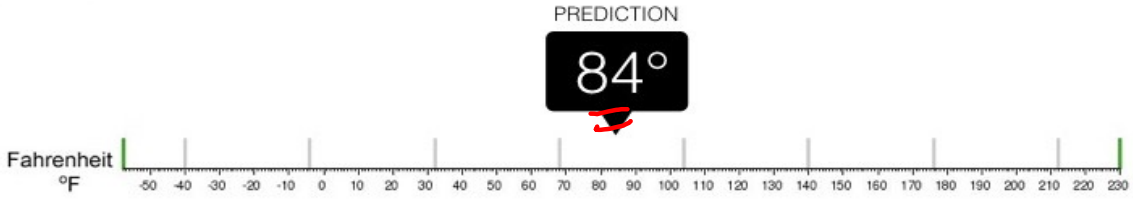
$\hat{y} = w_1 x_1 + w_2 x_2 + w_3$

$\begin{bmatrix} w_1 & w_2 & w_3 \\ - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$

Regression VS classification

Regression

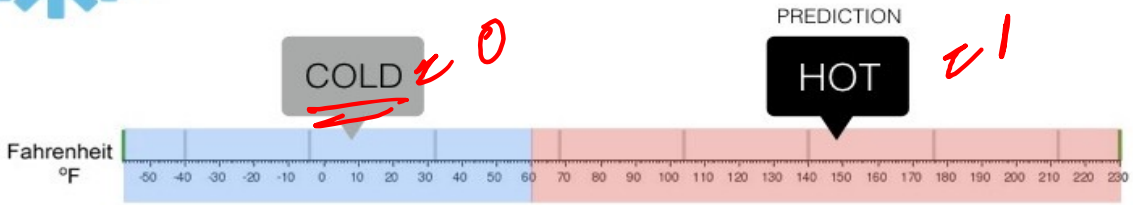
What is the temperature going to be tomorrow?



=> Continuous Values

Classification

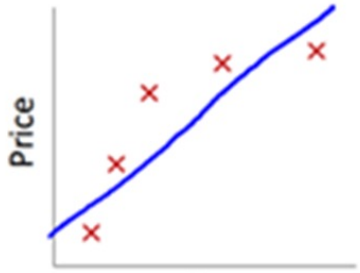
Will it be Cold or Hot tomorrow?



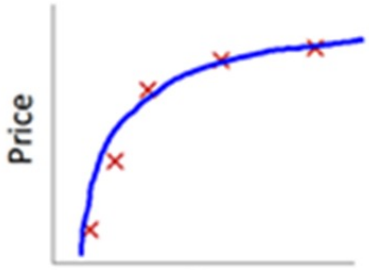
=> Discrete Values

Regression VS classification

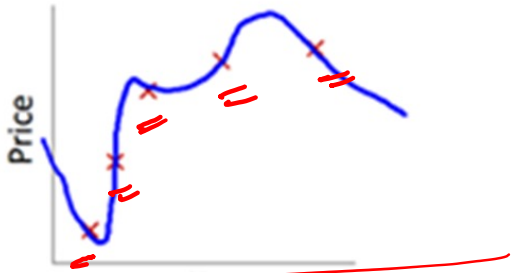
Regression:



Linear

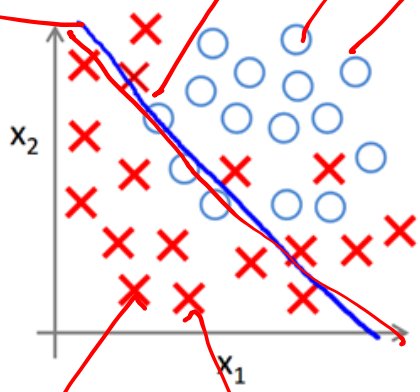


Polynomial

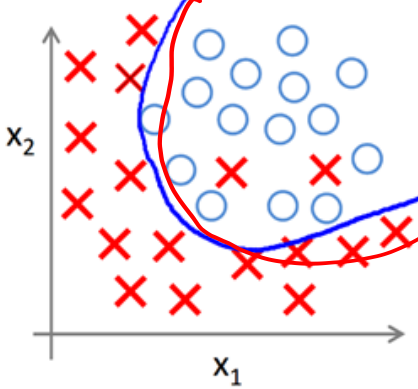


Very complex (overfitting)

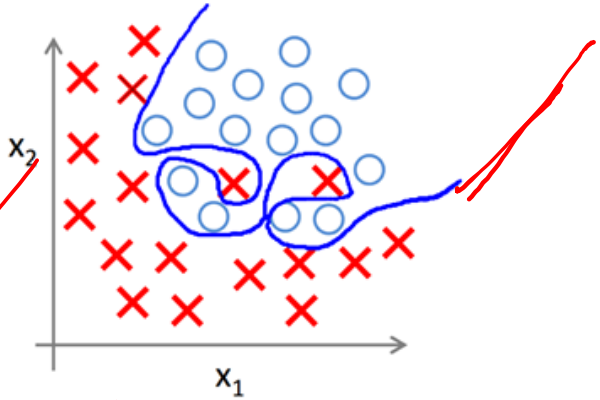
Classification:



Linear

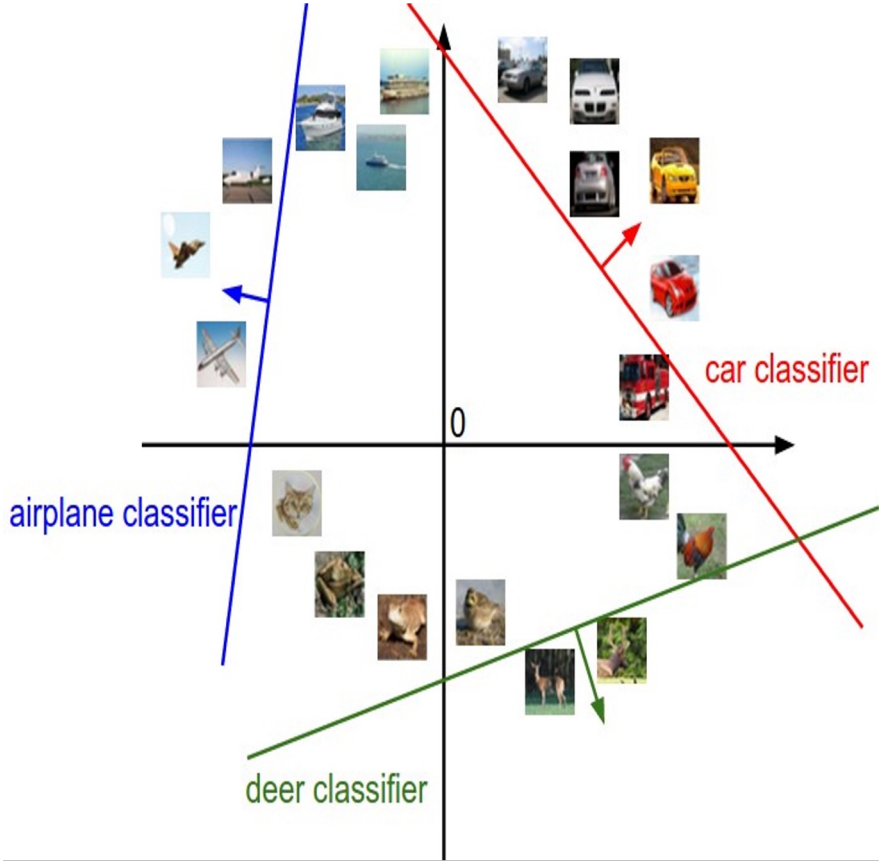


Polynomial



Very complex (overfitting)

Interpreting a Linear Classifier

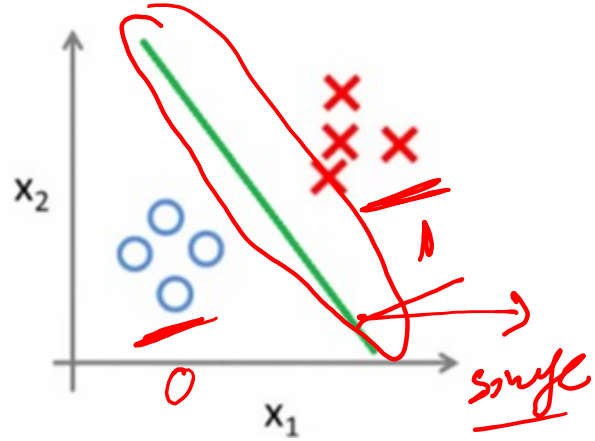


$$f(x_i, W, b) = Wx_i + b$$

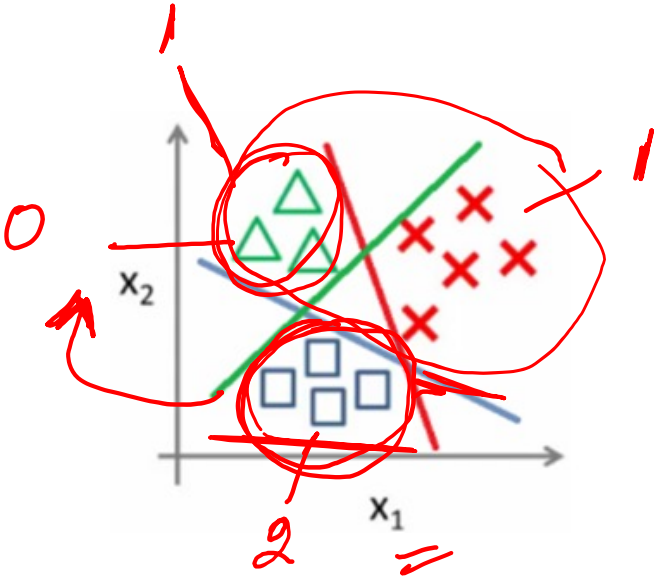
[32x32x3]
array of numbers 0...1
(3072 numbers total)

Classification Types

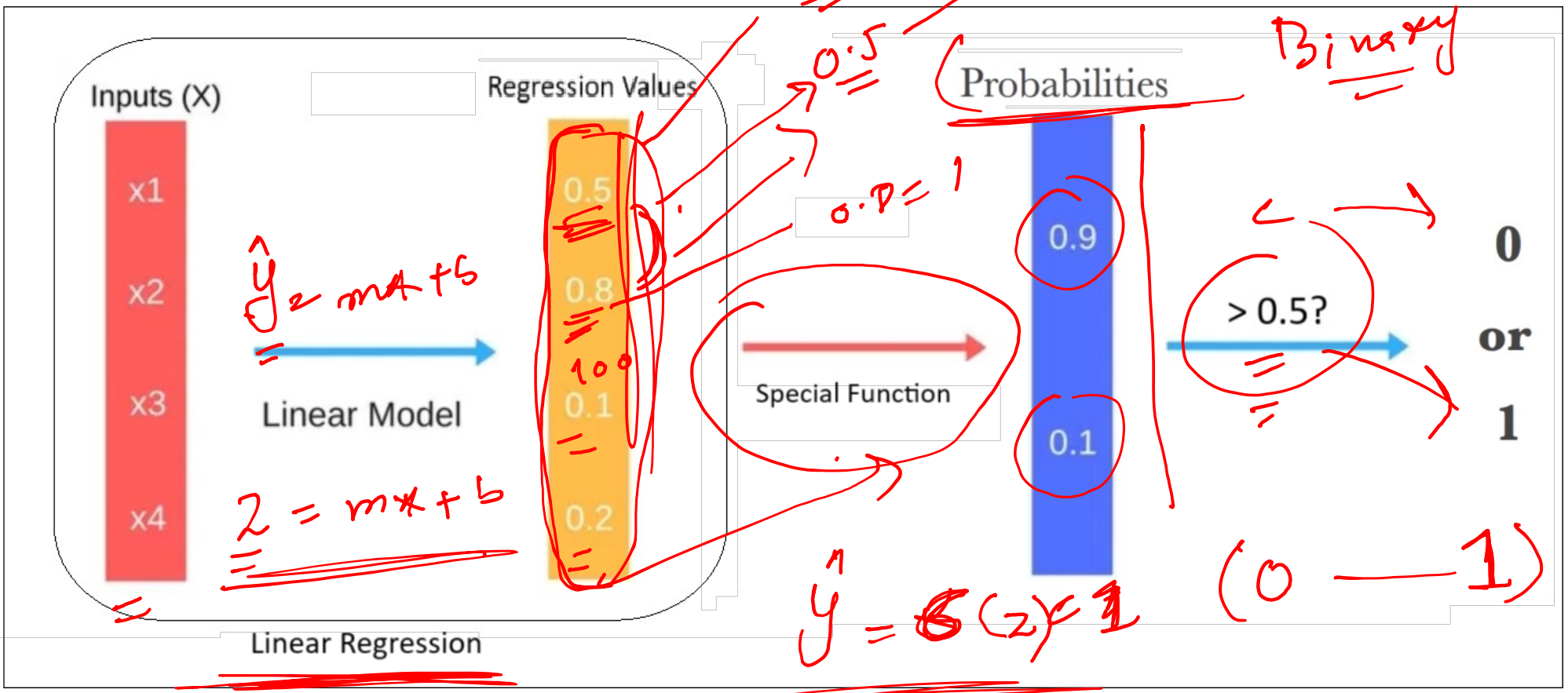
- **Binary Classification**: Two possible outcomes (e.g., Yes/No, Spam/Not Spam).
  How? Single Classifier.



- **Multiclass Classification**: More than two outcomes (e.g., Class A, B, C).
  How? Multiple *Single Classifiers*.



Logistic Regression

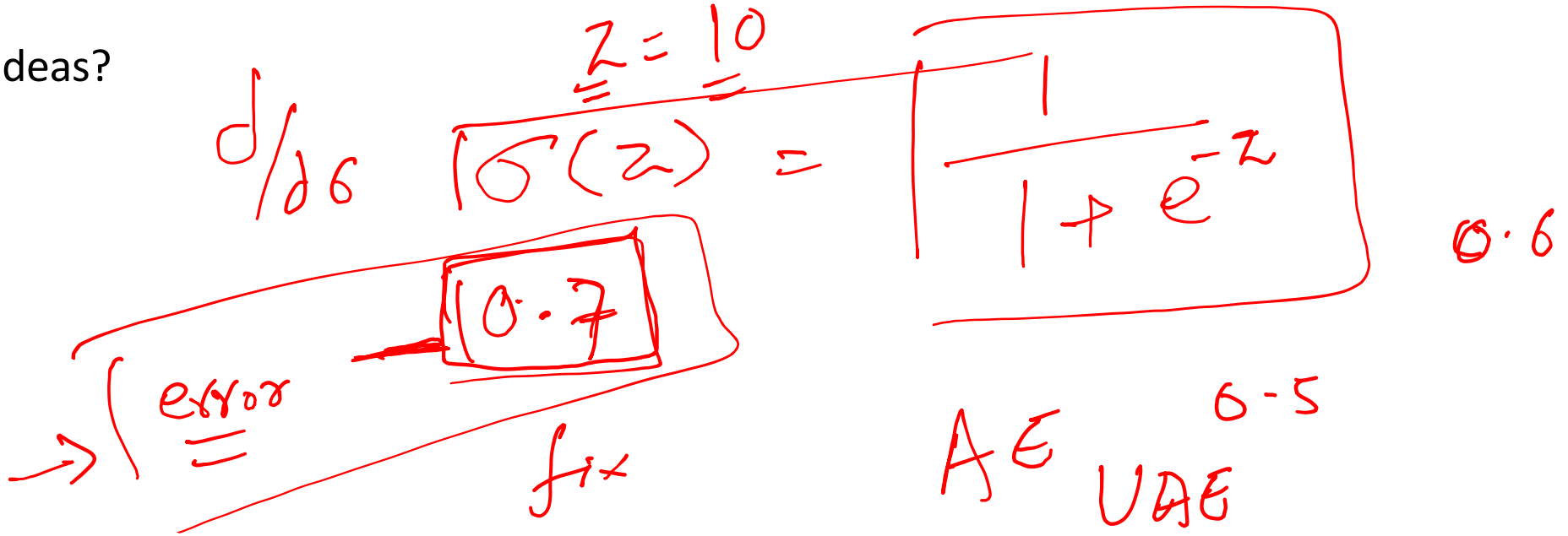


Special Function

We are searching for a function that has the following characteristics:

1. Could convert any arbitrary input values to **[0,1] range (Probabilities)**.
2. **Smooth and Differentiable**. Because we want to find the minima later, right?

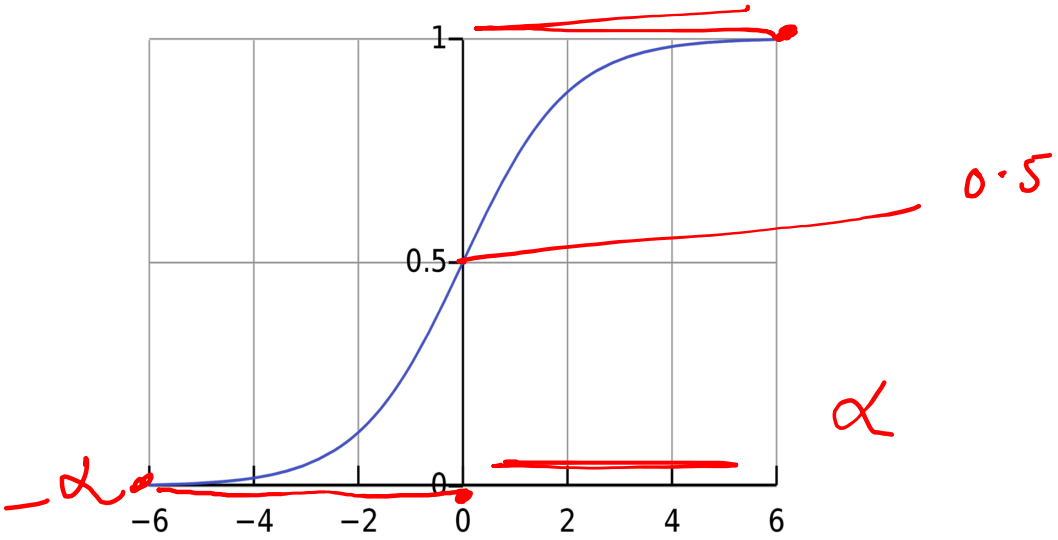
Any ideas?



Sigmoid Function (for Binary Classification)

$$\sigma(z) = \frac{1}{1 + exp(-z)}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



Assuming you have negative and positive classes, its output would be the probability of the positive class.

But what if we have multiclass problem?🤔

$$\lim_{z \to -\infty} \sigma(z) = 0$$

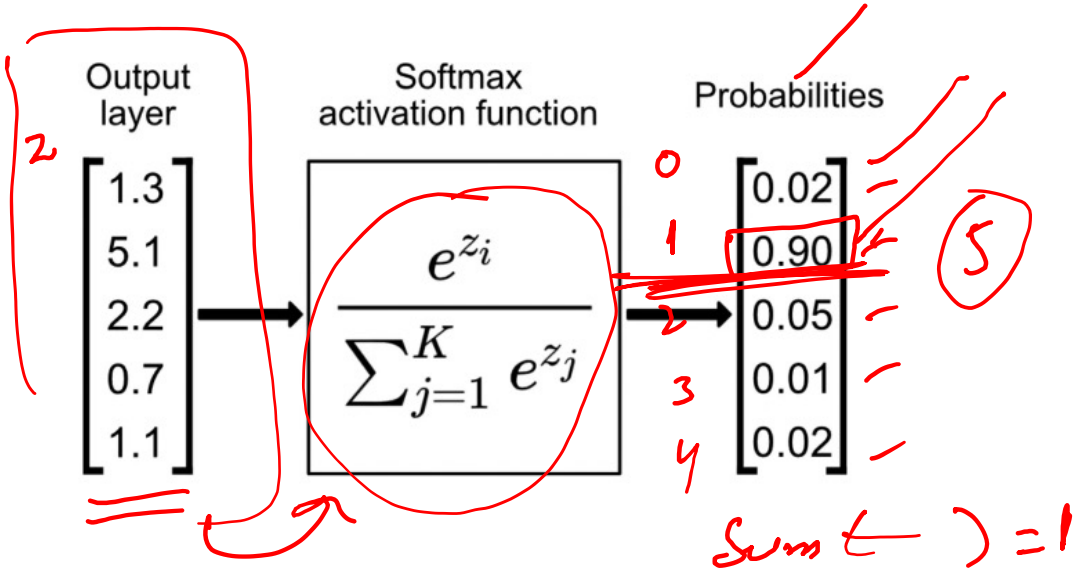
$$\lim_{z \to \infty} \sigma(z) = 1$$

Softmax Function (for Multiclass Classification)

$$Softmax(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{n} \exp(z_j)}$$

Numerator: The exponential of the ith class $\exp(z_i)$.
Denominator: The sum of the exponentials of all classes $\sum_{j=1}^{n} \exp(z_j)$.

E.g. Divide the cat value by the cat, dog and deer values to get the cat probability.



Assuming you have K classes, its output would be **the probability of the ith class**. So, you will run it K times to get the probability of each class.

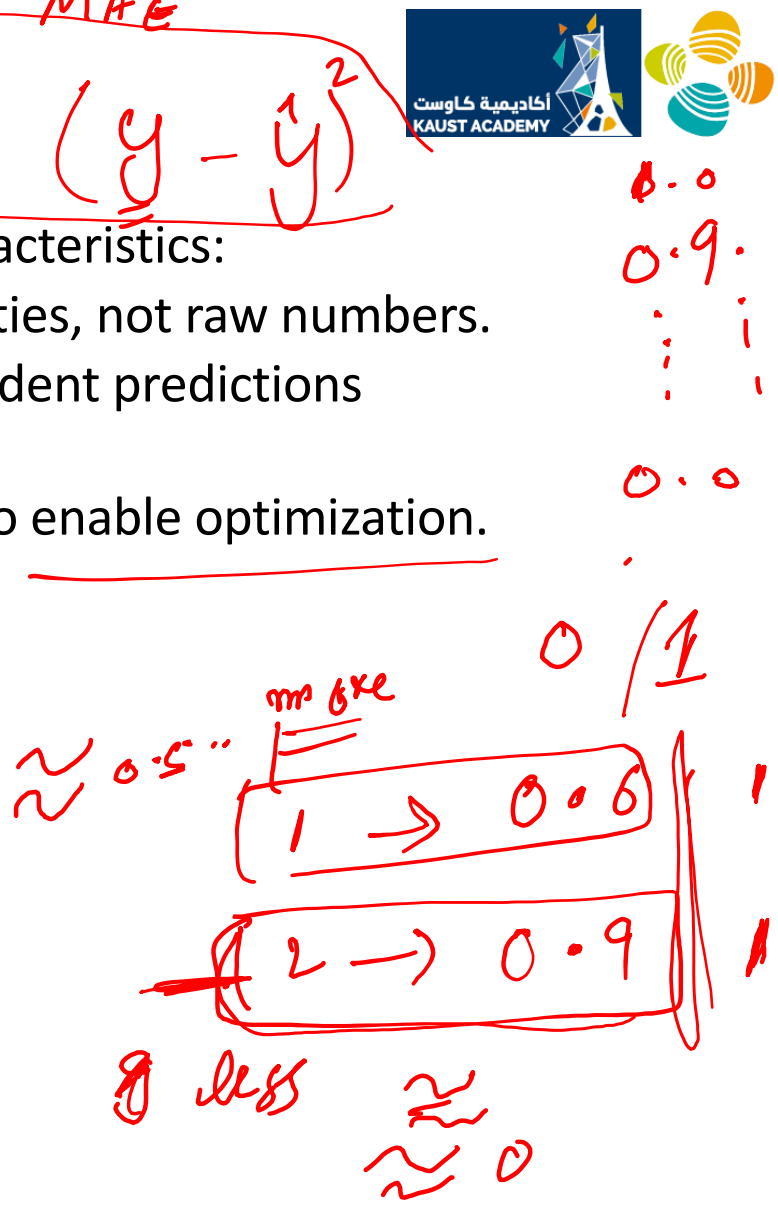
Note: Sigmoid is a special case of Softmax.
Optional: Can you prove it?

Classification Loss

We are searching for a loss that have the following characteristics:

1. **Probabilistic behaviour**: Should work with probabilities, not raw numbers.
2. **Sensitivity**: It should heavily penalize incorrect confident predictions (e.g., predicting 0.9 when the true label is 0).
3. **Differentiable**: Must be smooth and differentiable to enable optimization.

Any ideas?



Classification Loss

We are searching for a loss that have the following characteristics:

1. **Probabilistic behaviour**: Should work with probabilities, not raw numbers.
2. **Sensitivity**: It should heavily penalize incorrect confident predictions (e.g., predicting 0.9 when the true label is 0).
3. **Differentiable**: Must be smooth and differentiable to enable optimization.

Any ideas?

Logarithm is all you need :)

Binary Cross Entropy (LogLoss)

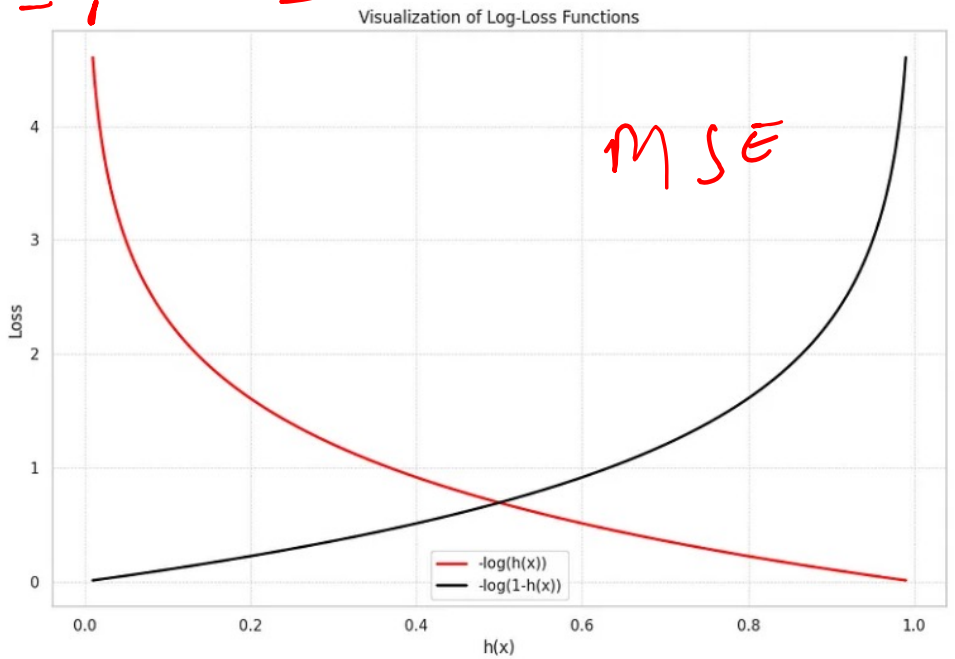
- For one sample, this can be represented mathematically by:

$$loss(y,p) = \begin{cases} -\log(p), & if\ y = 1 \\ -\log(1-p), & if\ y = 0 \end{cases}$$

Where y is the label, and p is the predicted probability.

- If $y = 1$ and prediction is close to 1, loss will be close to **zero**. (Great!)
- If $y = 1$ and prediction is close to 0, loss will be close to **infinity**. (Very bad!)

Same behavior for $y = 0$



Binary Cross Entropy (LogLoss)

- For one sample, this can be represented mathematically by:

$$loss(y,p) = \begin{cases} -\log(p), & if\ y = 1 \\ -\log(1-p), & if\ y = 0 \end{cases}$$

- Let's rewrite it in one line:

$$loss(y,p) = -(y * \log(p) + (1-y) * \log(1-p))$$

But we have many samples, not only one, right?

Binary Cross Entropy (LogLoss)

- For one sample, this can be represented mathematically by:

$$loss(y,p) = \begin{cases} -\log(p), & if\ y = 1 \\ -\log(1-p), & if\ y = 0 \end{cases}$$

- Let's write it in one line:

$$loss(y,p) = -(y * \log(p) + (1-y) * \log(1-p))$$

- Sum and average:

$$logloss(Y,P) = -\frac{1}{N}\sum_{i=1}^{N}(y_i * \log(p_i) + (1-y_i) * \log(1-p_i))$$

Y: Represents the true labels.

P: Represents the predicted probabilities for the positive class.

How to find optimal Parameters?

$$J(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\sigma(\mathbf{w}^T\mathbf{x}_i)) + (1-y_i)\log(1-\sigma(\mathbf{w}^T\mathbf{x}_i))$$

Just like before, simply take

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 0$$

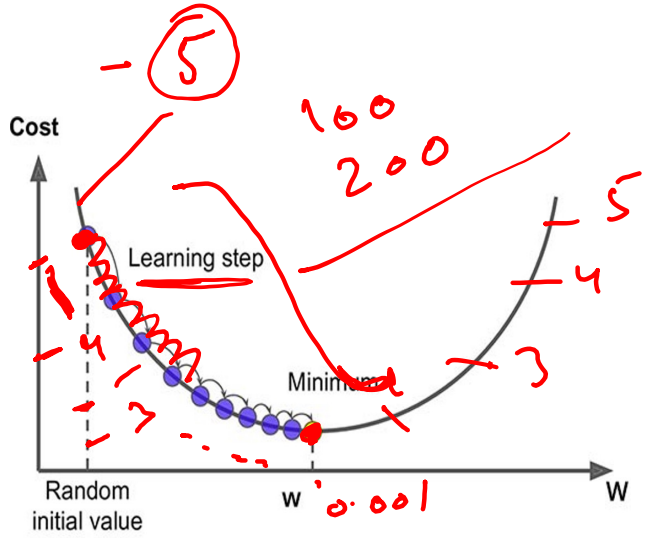
However, this does not have a nice closed solution, unlike the MSE case

How to find optimal Parameters?

$$J(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\sigma(\mathbf{w}^T\mathbf{x}_i)) + (1-y_i)\log(1-\sigma(\mathbf{w}^T\mathbf{x}_i))$$

Just like before, simply take

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 0$$



However, this does not have a nice closed solution unlike the MSE case

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^k)$$

Learning rate

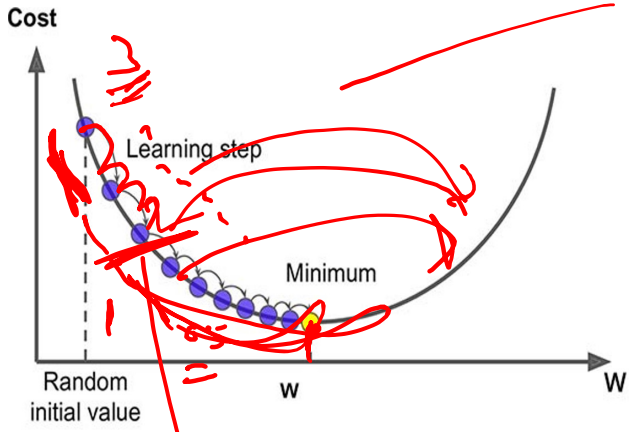
Instead, use gradient descent (optimizer)!

Logistic Regression

$$J(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\sigma(\mathbf{w}^T\mathbf{x}_i)) + (1-y_i)\log(1-\sigma(\mathbf{w}^T\mathbf{x}_i))$$

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta\nabla_{\mathbf{w}} J(\mathbf{w}^k)$$

Learning rate



- We have a linear model for prediction
- For classification, we want to output a probability
- We map the prediction to probabilities with a sigmoid function
- We have a loss function (BCE) to compare models

Logistic Regression

Linear Regression	Logistic Regression
For Regression	For Classification
We predict the target value for any input value	We predict the probability that the input value belongs to the specific target
Target: Real Values	Target: Discrete values
Graph: Straight Line	Graph: S-curve

Classification Metrics

- **Accuracy:**

  Accuracy measures the proportion of correctly classified samples out of the total number of samples.

$$Accuracy = \frac{Correct\ Samples}{All\ Sample}$$

Question: Why not optimizing for the accuracy directly instead of using LogLoss?

Classification Metrics

- **Accuracy:**

  Accuracy measures the proportion of correctly classified samples out of the total number of samples.

$$Accuracy = \frac{Correct\ Samples}{All\ Samples}$$

Question: Why not optimizing for the accuracy directly instead of using LogLoss?
Non-differentiable

Classification Metrics

- **Accuracy:**

  Accuracy measures the proportion of correctly classified samples out of the total number of samples.

$$Accuracy = \frac{Correct\ Samples}{All\ Sample}$$

Could also be written as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP: True Positives

TN: True Negatives

FP: False Positives

FN: False Negatives

Question: Why not optimizing for the accuracy directly instead of using LogLoss?
Non-differentiable

Classification Metrics

But accuracy isn’t always enough!

Imagine an imbalanced dataset with 95% negative labels (e.g., "not spam").

A model predicting "negative" all the time will be 95% accurate but useless.

Classification Metrics

But accuracy isn't always enough!

Imagine an imbalanced dataset with 95% negative labels (e.g., "not spam").
A model predicting "negative" all the time will be 95% accurate but useless.

Different goals require different metrics

- Do you care more about **catching all positives** (e.g., cancer detection)?
- Or avoiding **false alarms** (e.g., fraud detection)?

Classification Metrics

- **Precision:**

  Among all the positive predictions the model made,
  how many are actually correct?
  important when **minimizing false alarms** is critical,
  like in fraud detection or spam filtering.

$$Precision = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Positives\ (FP)}$$

Classification Metrics

- **Recall:**

  Among all the actual positives, how many did the model correctly identify?
  important when **catching all positives** is vital, such as in cancer detection.

$$Recall = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Negatives\ (FN)}$$

Classification Metrics

- **F1 Score:**

  What if both precision and recall are important?

  Take their (harmonic) mean.

$$F1\ score = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

Questions?